

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 627 — Theory of Computability

What no computer can do, proved — and why that is a practical fact rather than a philosophical one

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 8 (with CSCE 637 and CSCE 658)

Prerequisites: CSCE 629 Analysis of Algorithms; CSCE 605 Compiler Design is helpful for Modules 02–03 and 12; comfort with proof by contradiction and induction

Effort: 12–14 hours per week · 13 modules + 2 project weeks

Deliverable: A written portfolio of proofs — including one undecidability result you reduced yourself and one honest account of how an undecidable problem is handled in a tool you actually use

Course description

This is the only course in the program whose results are permanent. An algorithm can be improved and a model can be retrained; **a proof that no algorithm exists does not expire, and no amount of hardware or cleverness affects it.** That is worth a semester on its own, and it is why theory sits here rather than at the start.

The organising question is what a computation is. Module 01 takes that seriously, because the undecidability results of Modules 05 and 06 are only meaningful if 'algorithm' has a definition — and the fact that every reasonable definition turned out to be equivalent (Module 11) is the strongest evidence available that the definition is the right one.

The second theme is that the proofs are reusable techniques rather than isolated results. Diagonalisation (Module 05) and reduction (Module 06) are the two methods, and between them they establish essentially everything in the course — so learning them properly is learning the subject, and **reduction in particular is the same technique as CSCE 629's NP-hardness proofs.**

The third is that this material has immediate consequences. Module 08 catalogues the undecidable problems that appear in tools you use — type checking, alias analysis, termination, optimisation — and **Module 12 is about what engineering does instead:** soundness, completeness, and the deliberate choice of which to give up. **Every static analyser you have used embodies that choice,** and knowing which choice it made tells you what its output means.

And the fourth is that undecidability is not defeat. Compilers optimise, verifiers verify, and type checkers check, all while the underlying questions are undecidable — by restricting the input,

accepting approximation, or declining to answer. **Which is the engineering response, and it is a good one.**

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 State the Church–Turing thesis and explain what kind of claim it is.
- 2 Prove a language non-regular using the pumping lemma.
- 3 Place a language in the Chomsky hierarchy and justify it.
- 4 Explain Turing machine robustness and universality.
- 5 Distinguish decidable from recognisable and prove the halting problem undecidable.
- 6 Construct a mapping reduction to prove a problem undecidable.
- 7 State and apply Rice's theorem.
- 8 Place a problem in the arithmetic hierarchy.
- 9 Identify undecidable problems in real tools.
- 10 Explain Gödel's incompleteness via computability.
- 11 Explain the soundness-completeness trade in static analysis.
- 12 State what an undecidability result does and does not forbid.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	What a Computation Is	What does it mean to say something can be computed?
2	Finite Memory	What can be recognised with a fixed, finite amount of state?
3	Memory as a Stack	What does adding a stack buy you?
4	Turing Machines	What is the simplest model that computes everything computable?
5	Undecidability	Is there something no program can decide?
6	Reductions	How do you prove a <i>new</i> problem undecidable?
7	Degrees of Unsolvability	Are some undecidable problems harder than others?
8	Undecidability in the Wild	Where does undecidability actually show up?

Wk	Module	Core question it answers
9	Computable Functions	What is the structure of what <i>can</i> be computed?
10	Logic and Incompleteness	Can every true statement be proved?
11	Other Models	Why do all the definitions agree?
12	The Engineering Response	The question is undecidable. Now what?
13	The Limits, Honestly	What should you actually take from a semester of impossibility?

Projects

Project 1 — A portfolio of proofs (after Module 7)

Prove things, in writing, to a standard someone else could check.

Requirements

- **Two non-regularity proofs** by the pumping lemma, one of which needs a non-obvious decomposition.
- **One language shown context-free and one shown not**, with the argument in each case.
- **A Turing machine constructed and simulated** for a non-trivial task, with a trace.
- **The halting problem proved undecidable**, written out in your own words rather than reproduced.
- **Three mapping reductions** proving three different problems undecidable.
- **One application of Rice's theorem**, with the non-trivial-property condition verified.

Done means

- **Every proof checkable by someone else** without needing to ask you what a step meant.
- **The reductions in the right direction**, which is where most attempts fail — state explicitly what reduces to what.
- **The Rice condition actually verified**, not asserted.
- **At least one proof you got wrong first**, with the error and the fix both written down — which is graded and is the point.

Project 2 — Undecidability in a tool you use (after Module 12)

Take a real tool, find the undecidable problem it is pretending to solve, and characterise what it does instead.

Requirements

- **A tool you actually use**: a type checker, a linter, an optimiser, a verifier, a race detector.
- **The undecidable problem identified**, with a citation or a reduction showing it is undecidable.
- **The tool's strategy characterised**: is it sound, complete, or neither? Which did it give up?
- **A false positive you constructed**, with the source code.
- **A false negative you constructed**, with the source code.
- **The documentation's own claim quoted**, and assessed against what you found.

Done means

- **Both the false positive and the false negative working**, with code anyone can run.
- **The soundness position stated correctly** — most tools are neither sound nor complete, and saying so precisely is the exercise.

- **The undecidability argument correct**, not merely cited.
- **An honest assessment of whether the tool's own documentation is accurate about its limits**, which frequently it is not.

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
Sipser — Introduction to the Theory of Computation, 3rd edition	The primary source and the best-written textbook in theoretical computer science. Chapters 1–6 are this course. Library copy; the first chapter is free from the publisher and MIT 18.404 follows it exactly.
MIT 18.404J — Theory of Computation (free lectures and problem sets, taught by Sipser)	The author lecturing from his own book, free in full — video, notes, problem sets, and exams with solutions. This is the single best free resource in the program.
Hopcroft, Motwani & Ullman — Introduction to Automata Theory, Languages, and Computation	Stronger on Modules 02–03 than Sipser, and the parsing material connects directly to CSCE 605. Library copy.
Soare — Turing Computability; and Rogers — Theory of Recursive Functions	Module 07 and Module 09's reference — the arithmetic hierarchy and degrees, done properly. For the student who wants to go further.
Nagel & Newman — Gödel's Proof; and Smith — An Introduction to Gödel's Theorems	Module 10. Smith's book has a free study guide at this link, and it is the right level for someone who has just done Module 05.
Cousot & Cousot — Abstract Interpretation (free papers); and the Infer and Astrée documentation	Module 12's material — how production analysers live with undecidability, from the people who formalised the approach.

Tooling

- **Paper, and a willingness to write proofs out in full. This is the one course in the program where the deliverable is prose**, and a proof you cannot write down is a proof you do not have.
- **A simulator you write yourself** — a DFA, an NFA, a Turing machine. **Implementing the models makes the constructions concrete**, and the equivalence proofs of Module 02 become code you can test.
- **A SAT or SMT solver** (CSCE 625 Module 08) to see where the decidable boundary actually sits in practice — Z3 will cheerfully report 'unknown' on the undecidable fragments.
- **A static analyser on your own code**: a type checker in strict mode, a linter, Infer, or a termination checker. **Module 12 requires you to find a false positive and a false negative**, and both exist.
- **A proof assistant, optionally** — Lean or Coq. **Not required, and formalising one proof from this course is unusually instructive** about what a proof actually is.
- **No GPU, no cluster, and no dataset. The only course in the program with no computational requirement at all**, which is itself worth noticing.

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.